

IF2240 Basdat

Integrity Constraint

[Integrity Constraints](#)

[Constraint Definition](#)

[Type Constraints](#)

[Attribute Constraints](#)

[Relation Constraints](#)

[Database Constraints](#)

[State vs. Transition Constraints](#)

[Integrity in Relational Model](#)

[Keys: Candidate Keys](#)

[Keys: Primary Keys](#)

[Keys: Foreign Keys](#)

[Referential Actions](#)

Integrity Constraints

- Integrity refers to the consistency of data in the database
- Integrity constraints guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency
- A database might be subject to any number of integrity constraints, of arbitrary complexity.

For example:

- A checking account must have a balance greater than Rp10.000,00
- A salary of a bank employee must be at least Rp80.000,00 per hour
- A customer must have a phone number

Constraint Definition

- When a new constraint is defined, the system must make sure the database currently satisfies it.
 - If it doesn't, the new constraint will be rejected → you must modify existing data before enforcing the new constraint
 - Otherwise, it is accepted and enforced from that point forward
- Two approaches of constraint definitions:
 - **Declarative** integrity support → as part of system
 - **Procedural** integrity support → must be all declared in all application programs
- Example of constraint definitions:
CONSTRAINT SC3
IS_EMPTY (S WHERE STATUS < 1 OR STATUS > 100);
- Constraint examples:
 - An instructor name cannot be null
 - No two instructors can have the same instructor ID
 - every department name in the course relation must have a matching department name in the department relation
 - The budget of a department must be greater than zero.
- Constraint Classification Scheme
 - **Type constraint**
Specifies the legal values for a given type
 - **Attribute constraint**
specifies the legal values for a given attribute

- **Relation (relvar) constraint**
specifies the legal values for a given relation
- **Database constraint**
specifies the legal values for a given database

Type Constraints

- A type constraint is an enumeration of the legal values of the type
- Example:
TYPE WEIGHT POSSREP(RATIONAL)
CONSTRAINT THE_WEIGHT(WEIGHT) > 0.0;
- Type constraint can always be thought of as being checked during the execution of some selector invocation → type constraints are checked immediately

Attribute Constraints

- An attribute constraint is basically just a declaration to the effect that a specified attribute of a specified relation is of a specified type
- Example:
VAR S BASE RELATION(
S# S#,
SNAME SNAME, ...);
In this relation, values of attributes S#, SNAME, ... are constrained to be of types S#, SNAME, ..., respectively
- Attribute constraints are part of the definition of the attribute in question, and they can be identified by means of the corresponding attribute name

Relation Constraints

- A relation constraint is a constraint on an individual relation
- Example:
CONSTRAINT SC5
IS_EMPTY(S WHERE CITY = 'LONDON' AND STATUS <> 20);
- Relation constraints are checked immediately

Database Constraints

- A database constraint is a constraint that interrelates two or more distinct relations
- Example:

CONSTRAINT DBC1

IS_EMPTY ((S JOIN SP) WHERE STATUS < 20 AND QTY > QTY(500));

- In general, database constraint checking cannot be done immediately, but must be deferred to end of transaction. If a database constraint is violated at the end of transaction, the transaction is rolled back.

State vs. Transition Constraints

- State constraints concern with **consistent states** of the database
- Transition constraints concern with legal transitions from one correct state to another
- Examples:

Changes on marital status	
VALID TRANSITIONS: never married to married married to widowed widowed to married	INVALID TRANSITIONS: never married to widowed never married to divorced widowed to divorced
no supplier's status must ever decrease	
CONSTRAINT TRC1 IS_EMPTY (((S'(S#, STATUS) RENAME STATUS AS STATUS') JOIN S(S#, STATUS)) WHERE STATUS' > STATUS);	

- Transition constraints are only applied to **relation and database constraints**
 - A relation transition constraint can be checked immediately, while a database transition constraint would be deferred

Integrity in Relational Model

Keys: Candidate Keys

- Let K be a set of attributes of relation R. Then K is a candidate key for R iff it possesses both of the following properties:
 - Uniqueness: no legal value of R ever contains two distinct tuples with the same value for K
 - Irreducibility: no proper subset of K has the uniqueness property
- Candidate key provide the **basic tuple-level addressing mechanism**

Keys: Primary Keys

- A superset of a candidate key
- A candidate key that is selected to be the main key of the relation
- All the other candidate keys are called alternate keys.

Keys: Foreign Keys

- Let R2 be a relation. Then a foreign key in R2 is a set of attributes of R2, say FK, such that:
 - There exists a relation R1(not necessarily a distinct relation) with a candidate key CK, and
 - for all time, each value of FK in the current value of R2 is identical to the value of CK in some tuple in the current value of R1
- In practice, the corresponding candidate key usually is the primary key of R1
- The CK corresponding to some given foreign key might contain a value that doesn't currently appears as a value of that foreign key
- A foreign key can be simple or composite, depends to its matching CK
- A foreign key value represents a reference to the tuple containing the matching CK value(referenced tuple)
 - Referential integrity problem
 - Referential constraint
 - Referencing relation vs. Referenced relation
- The referential constraints among the relations in a database can be represented using **referential diagram**
- A relation can be both referencing and referenced. **referential path** the chain of arrows from one relation to another relation
- A relation is called **self-referencing** when its foreign key refer to its candidate key → referential cycle
- **Referential integrity rule:** The database must not contain any unmatched foreign key values

Referential Actions

- **Cascade:** the delete/update cascades to delete/update the tuples related
- **Restrict:** the delete/update is restricted to the ones that will not violate referential integrity constraints
- **No action:** the delete/update is performed exactly as requested